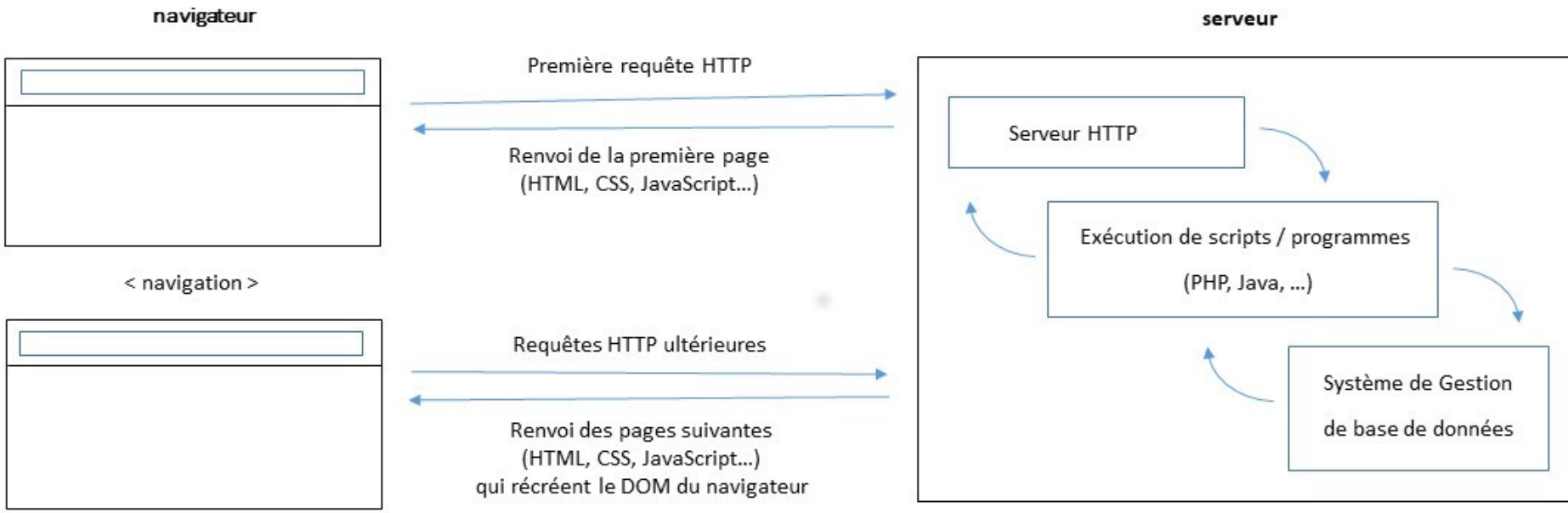
Technologies avancées du web

Architecture monopage (SPA)
Architecture hybride (CSR & SSR)
Architecture MEAN

Pierre Pompidor

Rappel de l'architecture multipages



Architectures monopages (Single Page application)

Chargement de toutes les vues (les codes HTML)
de l'application avant toute navigation

Architectures monopages (Single Page application)

Chargement de toutes les vues (les codes HTML) de l'application avant toute navigation

Chargement ultérieur des données

Architectures monopages (Single Page application)

Chargement de toutes les vues (les codes HTML) de l'application avant toute navigation

Chargement ultérieur des données

Optimisation des temps de navigation

Architectures monopages (Single Page application)

Chargement de toutes les vues (les codes HTML) de l'application avant toute navigation

Chargement ultérieur des données

Optimisation des temps de navigation

Premier temps de chargement plus long

Architectures monopages (Single Page application)

Chargement de toutes les vues (les codes HTML) de l'application avant toute navigation

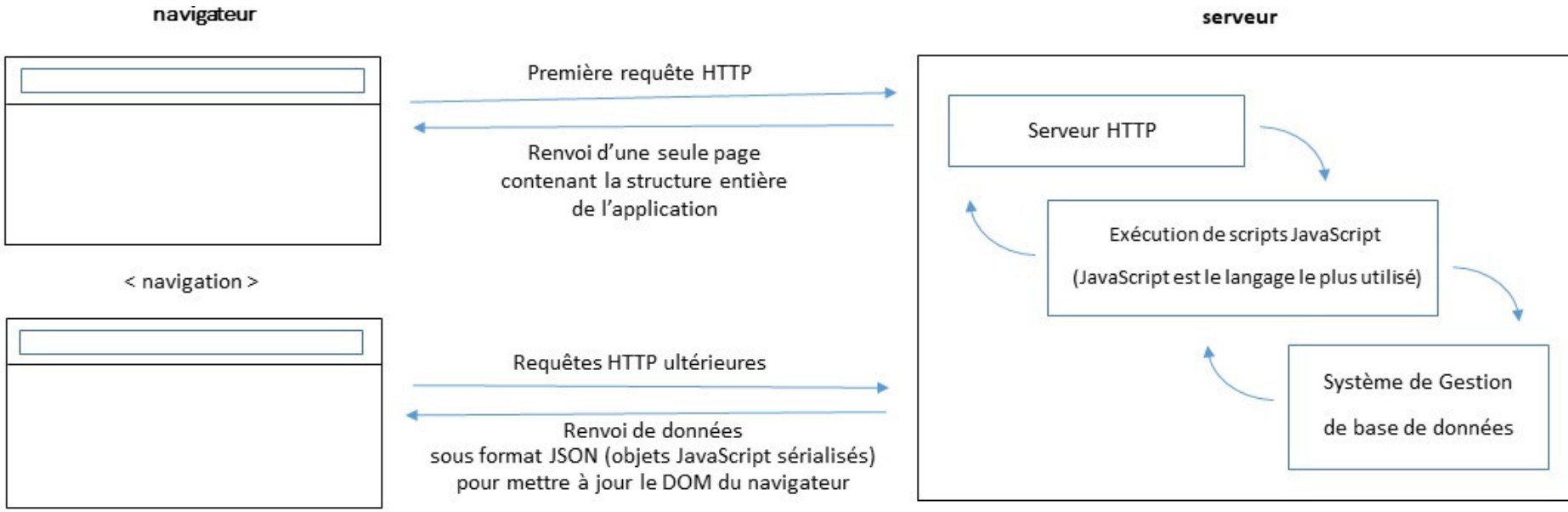
Chargement ultérieur des données

Optimisation des temps de navigation

Premier temps de chargement plus long

Difficulté du référencement (SEO *Search Engine Optimization*)

Architecture monopage (suite)



Architecture hybride

Pour garder le meilleur

de l'architecture multipages et monopage

Architecture hybride

Pour garder le meilleur

de l'architecture multipages et monopage

Des pages sont générées sur le serveur (SSR : Server-Side rendering)

- pour améliorer le référencement
- pour la mise en cache d'informations non dépendantes de la navigation

Architecture hybride

Pour garder le meilleur

de l'architecture multipages et monopage

Des pages sont générées sur le serveur (SSR : Server-Side rendering)

- pour améliorer le référencement
- pour la mise en cache d'informations non dépendantes de la navigation

Des composants sont hydratés du côté du client (CSR : Client-Side rendering)

- pour réduire les temps de navigation
- pour se découpler des fournisseurs de données

Architecture MEAN

MongoDB **E**xpress **A**ngular **N**ode.js ou **N**estJS

- Système de gestion de base de données noSQL **MongoDB**
- Serveur **Node.js** ou **NestJS** + framework REST **Express**
- Framework JavaScript Client **Angular**

Architecture MEAN

MongoDB **E**xpress **A**ngular **N**ode.js ou **N**estJS

- Système de gestion de base de données noSQL **MongoDB**
- Serveur **Node.js** ou **NestJS** + framework REST **Express**
- Framework JavaScript Client **Angular**

Avantages / Inconvénients

- +/- Utilisation unifiée de JavaScript (et TypeScript)
- + Gestion de grandes volumétries de données
- - Problématique du référencement (SEO)
  sauf en cas d'architecture hybride !

Système de gestion de bases de données MongoDB

SGBD noSQL

- **Collections** (= tables) de **Documents** (= enregistrements)
- Un document est un objet JavaScript "binarisé" (BSON)
- Recherches par filtres via un autre objet JavaScript

Système de gestion de bases de données MongoDB

SGBD noSQL

- **Collections** (= tables) de **Documents** (= enregistrements)
- Un document est un objet JavaScript "binarisé" (BSON)
- Recherches par filtres via un autre objet JavaScript

Avantages / Inconvénients

- - pas de vérification automatique de la cohérence des données (clefs étrangères)
- + très grande rapidité d'accès
  - point précédent
  - indexations multiples
- + gestion de grandes volumétries de données car possibilité de répartition de la base sur plusieurs machines physiques

Mise en oeuvre de MongoDB

Un service + un client

- Création d'un répertoire pour stocker les bases : **mkdir ~/path/mongodb/data**
- service : **mongod –dbpath ~/path/mongodb/data**
- client : **mongosh**

Mise en oeuvre de MongoDB

Un service + un client

- Création d'un répertoire pour stocker les bases : **mkdir ~/path/mongodb/data**
- service : **mongod –dbpath ~/path/mongodb/data**
- client : **mongosh**

Dans les salles de TP :

Si le service n'est pas déjà démarré : **mongod**

Utilisation de MongoDB dans le cadre des TP (application d'ecommerce)

- Création de trois collections sérialisées en JSON : articles.json, rayons.json et users.json
- Importation dans votre base via la commande **mongoimport**

Projet : création des collections users et articles

Collection **users**

```
[{"nom":"...", "prenom":"...", "email":"...",
  "password":"...", "civilite":"..."}, ...]
```

Projet : création des collections users et articles

Collection **users**

```
[{"nom":"...", "prenom":"...", "email":"...",
  "password":"...", "civilite":"..."}, ...]
```

Collection **rayons** : ["rayon":"...", "promotion":..., ...]

Collection **articles**

```
[{"nom":"...", "marque":"...", "prix":..., "descr":"...",
  "rayon":"...", "detail":"...", "stock":...}, ...]
```

Projet : création des collections users et articles

Collection **users**

```
[{"nom":"...", "prenom":"...", "email":"...",
  "password":"...", "civilite":"..."}, ...]
```

Collection **rayons** : ["rayon":"...", "promotion":..., ...]

Collection **articles**

```
[{"nom":"...", "marque":"...", "prix":..., "descr":"...",
  "rayon":"...", "detail":"...", "stock":...}, ...]
```

Importation avec mongoimport

mongoimport –db MEAN –collection users –file users.json –jsonArray –drop

idem pour les collections *rayons* et *articles*

MongoDB / mongosh : commandes de base

Bases et collections

- Voir les bases : **show dbs**
- Sélectionner une base : **use <base>**
- Voir les collections : **show collections** ou **show tables**

MongoDB / mongosh : commandes de base

Bases et collections

- Voir les bases : **show dbs**
- Sélectionner une base : **use <base>**
- Voir les collections : **show collections** ou **show tables**

Sélections et comptages dans la collection articles

```
SELECT * FROM articles : db.articles.find()

db.articles.find({rayon:'Telephonie'})

db.articles.find({rayon:'Telephonie',prix:{$gt:50,$lt:300}})

db.articles.countDocuments()

db.articles.countDocuments({rayon:'Telephonie'})

db.articles.distinct('rayon');
```